

Comprehensive Guide to C Programming

This guide provides a beginner-friendly, structured, and practical exploration of the C programming language, covering all topics from the syllabus. Each section includes a basic explanation of the concept, followed by examples with expected outputs to illustrate its application. The goal is to ensure a thorough understanding of C programming through clear explanations, practical examples, alternative approaches, and visible results.

Module 1: Introduction to C Programming

1.1 Course Overview: Title, Syllabus, Lesson Plan, and Evaluation Policy

Explanation:

This section introduces the course structure. The course, *Comprehensive C Programming*, teaches you how to write programs in C, a powerful language used in many applications. The syllabus lists all the topics we'll cover, like variables, loops, and file handling. The lesson plan ensures we learn step-by-step, and the evaluation policy explains how you'll be assessed (e.g., through assignments or quizzes).

Details:

- **Syllabus:** Includes C basics, control structures, arrays, pointers, functions, strings, file handling, data structures, and more.
- **Lesson Plan:** Each module builds on the previous one, with explanations and examples.
- **Evaluation Policy:** You'll be assessed through coding exercises, projects, and quizzes to test your understanding.

1.2 Problem Solving with C in the Era of AI

Explanation:

C is an older programming language, but it's still very important today, even with new technologies like artificial intelligence (AI). AI often needs fast programs, and C is great for speed because it runs directly on the computer's hardware. It's also used in small devices (like smartwatches) and helps you understand how computers work at a low level, which is useful for solving complex problems.

Applications:

- **Speed in AI:** AI tools like TensorFlow use C for fast calculations.
- **Small Devices:** C is used in devices like sensors because it doesn't need much memory.
- **Learning:** C teaches you how memory and programs work, making you a better problem-solver.

1.3 Salient Features of C

Explanation:

C has special qualities that make it a popular choice for programming:

- **Procedural:** You write C programs as a series of steps or functions, making it easy to follow.
- **Fast:** C programs turn into machine code that runs quickly on the computer.
- **Portable:** You can use the same C code on different computers with little change.
- **Low-Level Access:** C lets you control the computer's memory directly, which is powerful but needs care.
- **Modular:** You can break your program into smaller parts (functions) to reuse them.

1.4 Setting Up the Programming Environment

Explanation:

To write C programs, you need two tools: a text editor to write your code and a compiler to turn your code into a program the computer can run. A popular compiler for C is GCC (GNU Compiler Collection). You'll need to install it depending on your operating system (Windows, macOS, or Linux).

Steps:

- **Text Editor:** Use something simple like VS Code or Notepad++ to write your code.
- **Install GCC:**
 - **Windows:** Install MinGW-w64 (a version of GCC for Windows) and add its `bin` folder to your PATH.
 - **macOS:** Use the command `xcode-select --install` in the Terminal to get GCC.
 - **Linux:** Run `sudo apt update && sudo apt install build-essential` to install GCC.

1.5 Program Development Life Cycle (PDLC)

Explanation:

Writing a C program follows a cycle with these steps:

1. **Write:** Type your code in a file (e.g., `hello.c`).
2. **Compile:** Use a compiler (like GCC) to check for errors and turn your code into a program.
3. **Link:** Combine your code with built-in functions (like `printf`) to make a complete program.
4. **Execute:** Run the program to see the result.

Example:

- Write a file `hello.c`.
- Compile it: `gcc hello.c -o hello`.
- Run it: `./hello` (Linux/macOS) or `hello.exe` (Windows).

1.6 Program Structure, Characteristics, and Errors

Explanation:

A C program has a standard structure:

- It starts with `#include` to add libraries (like `stdio.h` for input/output).
 - The `main` function is where the program begins.
 - Inside `main`, you write your code, and it ends with `return 0` to say the program finished successfully.
- Errors can happen:
- **Syntax Errors:** Mistakes in writing, like missing a semicolon.
 - **Runtime Errors:** Problems when the program runs, like dividing by zero.
 - **Linker Errors:** Missing functions that were promised.
 - **Logical Errors:** The program runs but gives wrong results.

Example:

```
#include <stdio.h>
int main() {
    int x = 5;
    printf("Value of x: %d\n", x);
    return 0;
}
// Expected Output: Value of x: 5
```

- **Syntax Error Example:** `printf("Hello")` (missing `;`).
- **Runtime Error Example:** `int x = 0; printf("%d", 5/x);` (division by zero).
- **Logical Error Example:** `int avg = 5 / 2; printf("%d\n", avg);`
 - Expected Output: 2 (should be 2.5; fix with `5.0 / 2`).

1.7 Variables, Data Types, and Qualifiers

Explanation:

Variables are like boxes where you store data (e.g., numbers, letters). In C, you must say what type of data a variable holds:

- `int` : For whole numbers (e.g., 5, -10).
 - `char` : For single characters (e.g., 'A').
 - `float` / `double` : For decimal numbers (e.g., 3.14).
- Qualifiers change how variables work:
- `unsigned` : Makes numbers only positive, giving a bigger range.
 - `long` : Makes the variable store bigger numbers.

Data Types and Ranges:

Type	Size	Format Specifier	Range
<code>int</code>	4 bytes	<code>%d</code>	-2,147,483,648 to 2,147,483,647
<code>unsigned int</code>	4 bytes	<code>%u</code>	0 to 4,294,967,295
<code>char</code>	1 byte	<code>%c</code>	-128 to 127
<code>float</code>	4 bytes	<code>%f</code>	~6-7 decimal digits
<code>double</code>	8 bytes	<code>%lf</code>	~15-16 decimal digits

Example:

```
#include <stdio.h>
int main() {
    int age = 25;
    unsigned int count = 100;
    float price = 19.99;
    printf("Age: %d, Count: %u, Price: %.2f\n", age, count, price);
    return 0;
}
// Expected Output: Age: 25, Count: 100, Price: 19.99
```

1.8 Input/Output Functions

Explanation:

Input/output (I/O) functions let your program talk to the user.

- **Output (`printf`):** Shows text or numbers on the screen. You use format specifiers (like `%d` for integers) to show values.
- **Input (`scanf`):** Reads data from the user. You need to use `&` to tell `scanf` where to store the data.

- **Character I/O** (`getchar`, `putchar`): Reads or writes one character at a time.

Examples:

- **Output:**

```
#include <stdio.h>
int main() {
    int num = 42;
    printf("The number is %d\n", num);
    return 0;
}
// Expected Output: The number is 42
```

- **Input:**

```
#include <stdio.h>
int main() {
    int age;
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("You are %d years old\n", age);
    return 0;
}
// Input: 30
// Expected Output: Enter your age: You are 30 years old
```

- **Character I/O:**

```
#include <stdio.h>
int main() {
    char ch = getchar();
    putchar(ch);
    putchar('\n');
    return 0;
}
// Input: A
// Expected Output: A
```

1.9 Operators and Expressions

Explanation:

Operators are symbols that do things like math or comparisons:

- **Arithmetic:** `+`, `-`, `*`, `/`, `%` (remainder).
- **Relational:** Compare values, like `==` (equal), `>` (greater than).
- **Logical:** Combine conditions, like `&&` (and), `||` (or).
- **Bitwise:** Work on bits, like `&` (bitwise AND).
- **Ternary:** A shortcut for if-else, written as `condition ? value1 : value2`.
An expression is a combination of variables and operators that gives a result.

Examples:

- **Arithmetic:**

```
#include <stdio.h>
int main() {
    int sum = 5 + 3;
    printf("Sum: %d\n", sum);
    return 0;
}
// Expected Output: Sum: 8
```

- **Relational:**

```
#include <stdio.h>
int main() {
    int x = 10, y = 5;
    if (x > y) printf("x is greater\n");
    return 0;
}
// Expected Output: x is greater
```

- **Ternary:**

```
#include <stdio.h>
int main() {
    int x = 10, y = 5;
    int max = (x > y) ? x : y;
    printf("Max: %d\n", max);
    return 0;
}
// Expected Output: Max: 10
```

1.10 Language Specifications and Coding Standards

Explanation:

C has rules to avoid mistakes:

- **Undefined Behavior:** Some actions (like accessing an array beyond its size) can cause unpredictable results.
- **Coding Standards:** Rules to make your code readable:
 - Use `snake_case` for names (e.g., `my_variable`).
 - Indent your code (e.g., 4 spaces) to show structure.
 - Write comments to explain tricky parts.

Examples:

- **Undefined Behavior:**

```
#include <stdio.h>
int main() {
    int arr[3];
    arr[5] = 10; // Out of bounds
    printf("%d\n", arr[5]); // Undefined
    return 0;
}
// Expected Output: (Unpredictable, may crash)
```

- **Coding Standard:**

```
#include <stdio.h>
int main() {
    // Good: Clear variable name and indentation
    int total_score = 0;
    for (int i = 0; i < 5; i++) {
        total_score += i;
    }
    printf("Total: %d\n", total_score);
    return 0;
}
// Expected Output: Total: 10
```

Module 2: Control Structures

2.1 Selection Statements

Explanation:

Selection statements let your program make decisions:

- **if** : Runs code if a condition is true.

- **if-else** : Runs one block if true, another if false.
- **switch** : Chooses from multiple options based on a value. It's like a menu where you pick one option.

Examples:

- **if-else** :

```
#include <stdio.h>
int main() {
    int temp = 35;
    if (temp > 30) {
        printf("It's hot!\n");
    } else {
        printf("It's cool!\n");
    }
    return 0;
}
// Expected Output: It's hot!
```

- **switch** :

```
#include <stdio.h>
int main() {
    int choice = 2;
    switch (choice) {
        case 1: printf("Option 1\n"); break;
        case 2: printf("Option 2\n"); break;
        default: printf("Invalid\n"); break;
    }
    return 0;
}
// Expected Output: Option 2
```

2.2 Looping Statements

Explanation:

Loops repeat code until a condition is met:

- **for Loop**: Used when you know how many times to repeat (e.g., counting from 1 to 5).
- **while Loop**: Repeats as long as a condition is true.
- **do-while Loop**: Like `while`, but runs at least once because the condition is checked at the end.

Examples:

- **for** :

```
#include <stdio.h>
int main() {
    for (int i = 1; i <= 3; i++) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
// Expected Output: 1 2 3
```

- **while** :

```
#include <stdio.h>
int main() {
    int i = 1;
    while (i <= 3) {
        printf("%d ", i++);
    }
    printf("\n");
    return 0;
}
// Expected Output: 1 2 3
```

- **do-while :**

```
#include <stdio.h>
int main() {
    int choice = 1;
    do {
        printf("Loop iteration\n");
        choice = 0;
    } while (choice != 0);
    return 0;
}
// Expected Output: Loop iteration
```

2.3 Problem Solving Using Control Structures

Explanation:

Control structures help solve problems by making decisions or repeating tasks. For example, you can use a loop to sum numbers or an `if` statement to check conditions.

Example: Check if a number is even or odd.

```
#include <stdio.h>
int main() {
    int num = 4;
    if (num % 2 == 0) {
        printf("%d is even\n", num);
    } else {
        printf("%d is odd\n", num);
    }
    return 0;
}
// Expected Output: 4 is even
```

Single Character I/O Problem:

Explanation: You can use `getchar` and `putchar` to read and write one character at a time. This is useful for simple tasks like echoing user input.

Example: Echo characters until 'q'.

```
#include <stdio.h>
int main() {
    char ch;
    do {
        ch = getchar();
        putchar(ch);
    } while (ch != 'q');
    return 0;
}
// Input: abq
// Expected Output: abq
```

Module 3: Arrays, Pointers, and Functions

3.1 Arrays: 1D, Initialization, and Traversal

Explanation:

An array is a list of items of the same type (e.g., all integers). A 1D array is a single list. You can set values when you create it (initialization) and go through each item (traversal) using a loop.

Example:

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50}; // Initialization
    for (int i = 0; i < 5; i++) { // Traversal
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
// Expected Output: 10 20 30 40 50
```

3.2 Arrays: 2D

Explanation:

A 2D array is like a grid (rows and columns), useful for things like tables or matrices. You need two loops to go through it: one for rows, one for columns.

Example:

```
#include <stdio.h>
int main() {
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
// Expected Output:
// 1 2 3
// 4 5 6
```

3.3 Problem Solving Using Arrays

Explanation:

Arrays are great for solving problems involving lists, like finding the largest number or calculating an average.

Example: Find the average of numbers.

```
#include <stdio.h>
int main() {
    int arr[4] = {10, 20, 30, 40};
    int sum = 0;
    for (int i = 0; i < 4; i++) {
        sum += arr[i];
    }
    printf("Average: %.2f\n", (float)sum / 4);
    return 0;
}
// Expected Output: Average: 25.00
```

3.4 Pointers

Explanation:

A pointer is a variable that holds a memory address. It's like a map pointing to where data is stored. You use `*` to get the value at that address (dereferencing) and `&` to get the address of a variable.

Example:

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x; // p holds the address of x
    printf("Value: %d\n", *p); // Dereference to get value
    *p = 20; // Changes x to 20
    printf("New value of x: %d\n", x);
    return 0;
}
// Expected Output:
// Value: 10
// New value of x: 20
```

3.5 Array of Pointers

Explanation:

An array of pointers is a list where each item is a pointer. It's often used to store multiple strings, where each pointer points to the start of a string.

Example:

```
#include <stdio.h>
int main() {
    const char *names[] = {"Alice", "Bob", "Charlie", NULL};
    for (int i = 0; names[i]; i++) {
        printf("%s\n", names[i]);
    }
    return 0;
}
// Expected Output:
// Alice
// Bob
// Charlie
```

3.6 Functions

Explanation:

A function is a block of code that does a specific task, like adding numbers. You can call it whenever you need that task done, making your code reusable and organized.

Example:

```
#include <stdio.h>
int add(int a, int b) {
    return a + b;
}
int main() {
    int sum = add(5, 3);
    printf("Sum: %d\n", sum);
    return 0;
}
// Expected Output: Sum: 8
```

3.7 Arrays and Functions

Explanation:

When you pass an array to a function, you're actually passing a pointer to its first element. This means the function can change the original array.

Example:


```
#include <stdio.h>

void double_array(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2;
    }
}

int main() {
    int arr[] = {1, 2, 3};
    double_array(arr, 3);
    for (int i = 0; i < 3; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}

// Expected Output: 2 4 6
```

3.8 Problem Solving Using Arrays, Functions, and Pointers

Explanation:

You can combine arrays, functions, and pointers to solve problems efficiently. For example, you can use pointers to swap array elements without extra variables.

Example: Reverse an array using pointers.

```
#include <stdio.h>

void reverse(int *arr, int size) {
    int *start = arr, *end = arr + size - 1;
    while (start < end) {
        int temp = *start;
        *start++ = *end;
        *end-- = temp;
    }
}

int main() {
    int arr[] = {1, 2, 3, 4};
    reverse(arr, 4);
    for (int i = 0; i < 4; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}

// Expected Output: 4 3 2 1
```

3.9 Storage Classes

Explanation:

Storage classes decide how long a variable lives and where it can be used:

- **auto** : Default for local variables; they disappear when the function ends.
- **static** : Keeps its value between function calls.
- **extern** : Shares a variable across different files.
- **register** : Suggests storing the variable in a CPU register for speed.

Example (static):

```
#include <stdio.h>
void count_calls() {
    static int count = 0; // Retains value
    count++;
    printf("Called %d times\n", count);
}
int main() {
    count_calls();
    count_calls();
    return 0;
}
// Expected Output:
// Called 1 times
// Called 2 times
```

3.10 Recursion

Explanation:

Recursion is when a function calls itself to solve a problem. It needs a base case (to stop) and a recursive step (to break the problem into smaller parts). It's useful for problems like calculating factorials.

Example: Factorial.

```
#include <stdio.h>
int factorial(int n) {
    if (n <= 1) return 1; // Base case
    return n * factorial(n - 1); // Recursive step
}
int main() {
    printf("Factorial of 4: %d\n", factorial(4));
    return 0;
}
// Expected Output: Factorial of 4: 24
```

Problem Solving: Sum of digits.

```
#include <stdio.h>
int sum_digits(int n) {
    if (n == 0) return 0;
    return (n % 10) + sum_digits(n / 10);
}
int main() {
    printf("Sum of digits in 123: %d\n", sum_digits(123));
    return 0;
}
// Expected Output: Sum of digits in 123: 6
```

3.11 Callbacks Using Function Pointers

Explanation:

A function pointer is a pointer that points to a function. A callback is when you pass a function as an argument to another function, so it can be called later. This makes your code flexible, like letting a function decide how to process data.

Example: Apply a function to a number.

```
#include <stdio.h>
typedef int (*Operation)(int);
int square(int x) { return x * x; }
void apply(int x, Operation op) {
    printf("Result: %d\n", op(x));
}
int main() {
    apply(4, square);
    return 0;
}
// Expected Output: Result: 16
```

Problem Solving (Counting): Count numbers greater than 5.

```
#include <stdio.h>
int greater_than_5(int x) { return x > 5; }
int count_if(int arr[], int size, int (*condition)(int)) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (condition(arr[i])) count++;
    }
    return count;
}
int main() {
    int arr[] = {3, 7, 1, 9, 4};
    printf("Numbers > 5: %d\n", count_if(arr, 5, greater_than_5));
    return 0;
}
// Expected Output: Numbers > 5: 2
```

Module 4: Searching and Sorting

4.1 Searching

Explanation:

Searching means finding an item in a list:

- **Linear Search:** Check each item one by one until you find the target or reach the end. It works on any list.
- **Binary Search:** Works only on sorted lists. It divides the list in half each time, making it much faster.

Examples:

- **Linear Search:**

```
#include <stdio.h>
int linear_search(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
int main() {
    int arr[] = {5, 2, 8, 1};
    int pos = linear_search(arr, 4, 8);
    printf("Position: %d\n", pos);
    return 0;
}
// Expected Output: Position: 2
```

- **Binary Search:**

```

#include <stdio.h>
int binary_search(int arr[], int size, int target) {
    int low = 0, high = size - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
int main() {
    int arr[] = {1, 3, 5, 7};
    int pos = binary_search(arr, 4, 5);
    printf("Position: %d\n", pos);
    return 0;
}
// Expected Output: Position: 2

```

Alternative (Recursive Linear Search):

```

#include <stdio.h>
int linear_search_recursive(int arr[], int size, int target, int index) {
    if (index >= size) return -1;
    if (arr[index] == target) return index;
    return linear_search_recursive(arr, size, target, index + 1);
}
int main() {
    int arr[] = {5, 2, 8, 1};
    int pos = linear_search_recursive(arr, 4, 8, 0);
    printf("Position: %d\n", pos);
    return 0;
}
// Expected Output: Position: 2

```

4.2 Sorting

Explanation:

Sorting arranges items in order (e.g., smallest to largest). C has a built-in function called `qsort` that sorts lists quickly. You tell `qsort` how to compare items using a callback function.

Example: Sort numbers.

```

#include <stdio.h>
#include <stdlib.h>
int compare(const void *a, const void *b) {
    return *(int*)a - *(int*)b;
}
int main() {
    int arr[] = {4, 1, 3, 2};
    qsort(arr, 4, sizeof(int), compare);
    for (int i = 0; i < 4; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
// Expected Output: 1 2 3 4

```

Sorting with Callbacks and Array of Pointers: Sort strings.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int compare_strings(const void *a, const void *b) {
    return strcmp(*(const char**)a, *(const char**)b);
}
int main() {
    const char *names[] = {"Bob", "Alice", "Charlie"};
    qsort(names, 3, sizeof(const char*), compare_strings);
    for (int i = 0; i < 3; i++) printf("%s ", names[i]);
    printf("\n");
    return 0;
}
// Expected Output: Alice Bob Charlie

```

Problem Solving with Callbacks (Searching):

```

#include <stdio.h>
int is_even(int x) { return x % 2 == 0; }
int search_if(int arr[], int size, int (*condition)(int)) {
    for (int i = 0; i < size; i++) {
        if (condition(arr[i])) return i;
    }
    return -1;
}
int main() {
    int arr[] = {1, 3, 4, 7, 8};
    int pos = search_if(arr, 5, is_even);
    printf("First even at: %d\n", pos);
    return 0;
}
// Expected Output: First even at: 2

```

Module 5: Strings and Command Line Arguments

5.1 Strings in C

Explanation:

In C, a string is a list of characters (like "Hello") ending with a special character `\0` (null terminator). This `\0` tells the computer where the string ends. You can create strings in different ways: using an array, a pointer, or by allocating memory dynamically.

Initialization Examples:

- **Array:**

```

#include <stdio.h>
int main() {
    char str[] = "Hello";
    printf("%s\n", str);
    return 0;
}
// Expected Output: Hello

```

- **Pointer (Read-Only):**

```

#include <stdio.h>
int main() {
    const char *str = "Hello";
    printf("%s\n", str);
    return 0;
}
// Expected Output: Hello

```

- **Dynamic Allocation:**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main() {
    char *str = (char*)malloc(10);
    strcpy(str, "Hello");
    printf("%s\n", str);
    free(str);
    return 0;
}
// Expected Output: Hello
```

5.2 String Manipulation Functions

Explanation:

The `<string.h>` library has functions to work with strings, like finding their length, copying them, or comparing them. Here's a list of the most common ones:

- `strlen`: Counts characters (not including `\0`).
- `strcmp`: Compares two strings to see which comes first alphabetically.
- `strncmp`: Compares only the first `n` characters.
- `strcpy`: Copies one string into another.
- `strncpy`: Copies up to `n` characters (safer).
- `strcat`: Adds one string to the end of another.
- `strncat`: Adds up to `n` characters.
- `strchr`: Finds a character in a string.
- `strrchr`: Finds the last occurrence of a character.
- `strstr`: Finds a substring.
- `strtok`: Splits a string into pieces (tokens).
- `strspn`: Counts how many starting characters are in a set.
- `strcspn`: Counts how many starting characters are not in a set.
- `strpbrk`: Finds the first character that matches any in a set.

Examples:

- `strlen`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello";
    printf("Length: %zu\n", strlen(str));
    return 0;
}
// Expected Output: Length: 5
```

- `strcmp`:

```
#include <stdio.h>
#include <string.h>
int main() {
    if (strcmp("apple", "banana") < 0) printf("apple comes first\n");
    return 0;
}
// Expected Output: apple comes first
```

- `strncmp`:

```
#include <stdio.h>
#include <string.h>
int main() {
    if (strncmp("apple", "apricot", 3) == 0) printf("First 3 chars match\n");
    return 0;
}
// Expected Output: First 3 chars match
```

- `strcpy`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest[10];
    strcpy(dest, "Hello");
    printf("%s\n", dest);
    return 0;
}
// Expected Output: Hello
```

- `strncpy`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest[10] = {0};
    strncpy(dest, "Hello", 5);
    printf("%s\n", dest);
    return 0;
}
// Expected Output: Hello
```

- `strcat`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest[20] = "Hello";
    strcat(dest, " World");
    printf("%s\n", dest);
    return 0;
}
// Expected Output: Hello World
```

- `strncat`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest[20] = "Hello";
    strncat(dest, " World", 6);
    printf("%s\n", dest);
    return 0;
}
// Expected Output: Hello World
```

- `strchr`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char *pos = strchr("Hello", 'l');
    printf("First 'l' at: %ld\n", pos - "Hello");
    return 0;
}
// Expected Output: First 'l' at: 2
```

- `strrchr`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char *pos = strrchr("Hello", 'l');
    printf("Last 'l' at: %ld\n", pos - "Hello");
    return 0;
}
// Expected Output: Last 'l' at: 3
```

- `strstr`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char *pos = strstr("Hello World", "World");
    printf("'World' at: %ld\n", pos - "Hello World");
    return 0;
}
// Expected Output: 'World' at: 6
```

- `strtok`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "apple,banana,orange";
    char *token = strtok(str, ",");
    while (token) {
        printf("%s\n", token);
        token = strtok(NULL, ",");
    }
    return 0;
}
// Expected Output:
// apple
// banana
// orange
```

- `strspn`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "123abc";
    printf("Numeric prefix: %zu\n", strspn(str, "0123456789"));
    return 0;
}
// Expected Output: Numeric prefix: 3
```

- `strcspn`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello\nWorld";
    printf("Before newline: %zu\n", strcspn(str, "\n"));
    return 0;
}
// Expected Output: Before newline: 5
```

- `strpbrk`:


```
#include <stdio.h>
#include <string.h>
int main() {
    char *pos = strpbrk("Hello", "aeiou");
    printf("First vowel at: %ld\n", pos - "Hello");
    return 0;
}
// Expected Output: First vowel at: 1
```

5.3 Errors and Best Practices

Explanation:

Strings can cause problems if not handled carefully:

- **Buffer Overflow:** Writing more data than the string can hold (e.g., copying "Hello" into a 3-byte array).
- **Missing `\0`:** Some functions don't add the null terminator, causing errors.

Best Practices:

- Use `fgets` to read strings safely.
- Use `snprintf` to format strings without overflow.
- Always check that your array is big enough.

Examples:

- **Buffer Overflow:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest[3];
    strcpy(dest, "Hello"); // Error: Needs 6 bytes
    printf("%s\n", dest); // Undefined
    return 0;
}
// Expected Output: (Unpredictable, may crash)
```

- **Safe Input with `fgets`:**

```
#include <stdio.h>
#include <string.h>
int main() {
    char name[50];
    printf("Enter name: ");
    fgets(name, 50, stdin);
    name[strcspn(name, "\n")] = '\0'; // Remove newline
    printf("Hello, %s\n", name);
    return 0;
}
// Input: Alice
// Expected Output:
// Enter name: Hello, Alice
```

5.4 Command Line Arguments

Explanation:

When you run a C program from the command line, you can give it extra information (arguments). These are passed to the `main` function as `argc` (number of arguments) and `argv` (array of strings).

Example:

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <name>\n", argv[0]);
        return 1;
    }
    printf("Hello, %s!\n", argv[1]);
    return 0;
}

// Run: ./program Alice
// Expected Output: Hello, Alice!
```

Problem Solving: Calculator.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 4) return 1;
    double a = atof(argv[1]), b = atof(argv[3]);
    char op = argv[2][0];
    double result;
    switch (op) {
        case '+': result = a + b; break;
        case '-': result = a - b; break;
        default: return 1;
    }
    printf("Result: %.2f\n", result);
    return 0;
}

// Run: ./program 5 + 3
// Expected Output: Result: 8.00
```

Module 6: Dynamic Memory Management

6.1 Dynamic Memory Management

Explanation:

Normally, variables in C have a fixed size when you write the program. But sometimes you don't know how much space you need until the program runs. Dynamic memory lets you ask for space while the program is running, using functions like `malloc` and `free`.

- `malloc`: Gives you a block of memory.
- `calloc`: Like `malloc`, but sets the memory to zero.
- `realloc`: Changes the size of memory you already have.
- `free`: Gives the memory back when you're done.

Examples:

- `malloc`:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int*)malloc(3 * sizeof(int));
    if (!arr) return 1;
    arr[0] = 10; arr[1] = 20; arr[2] = 30;
    for (int i = 0; i < 3; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}

// Expected Output: 10 20 30
```

- `calloc`:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *arr = (int*)calloc(3, sizeof(int));
    if (!arr) return 1;
    for (int i = 0; i < 3; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}
// Expected Output: 0 0 0
```

- `realloc`:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *arr = (int*)malloc(2 * sizeof(int));
    arr[0] = 1; arr[1] = 2;
    arr = (int*)realloc(arr, 4 * sizeof(int));
    if (!arr) return 1;
    arr[2] = 3; arr[3] = 4;
    for (int i = 0; i < 4; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}
// Expected Output: 1 2 3 4
```

Error Handling: Always check if `malloc` returns `NULL` (means it failed), and always `free` memory to avoid leaks.

6.2 Simple Linked List

Explanation:

A linked list is a way to store items where each item (called a node) points to the next one. Unlike arrays, linked lists can grow or shrink easily because they don't need to be in one block of memory.

Example:

```

#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *next;
} Node;
void insert(Node **head, int data) {
    Node *new_node = (Node*)malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = *head;
    *head = new_node;
}
void print(Node *head) {
    while (head) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}
int main() {
    Node *head = NULL;
    insert(&head, 3);
    insert(&head, 2);
    print(head);
    return 0;
}
// Expected Output: 2 -> 3 -> NULL

```

6.3 Stack

Explanation:

A stack is a list where you add (push) and remove (pop) items from the same end, like a stack of plates (last in, first out, or LIFO). It's useful for things like undoing actions in a program.

Example:

```

#include <stdio.h>
#define SIZE 5
int stack[SIZE], top = -1;
void push(int val) {
    if (top == SIZE - 1) return;
    stack[++top] = val;
}
int pop() {
    if (top == -1) return -1;
    return stack[top--];
}
int main() {
    push(10);
    push(20);
    printf("Popped: %d\n", pop());
    return 0;
}
// Expected Output: Popped: 20

```

6.4 Queue and Priority Queue

Explanation:

A queue is a list where you add items at the end (enqueue) and remove them from the front (dequeue), like a line at a store (first in, first out, or FIFO). A priority queue is a special queue where items with higher priority (e.g., more urgent) are removed first, regardless of order.

Example (Queue):

```

#include <stdio.h>
#define SIZE 5
int queue[SIZE], front = 0, rear = -1;
void enqueue(int val) {
    if (rear == SIZE - 1) return;
    queue[++rear] = val;
}
int dequeue() {
    if (front > rear) return -1;
    return queue[front++];
}
int main() {
    enqueue(10);
    enqueue(20);
    printf("Dequeued: %d\n", dequeue());
    return 0;
}
// Expected Output: Dequeued: 10

```

Module 7: User-Defined Types

7.1 Structures and Functions

Explanation:

A structure (`struct`) lets you group different types of data together, like a student's name and ID. You can pass structures to functions to work with them.

Example:

```

#include <stdio.h>
typedef struct {
    char name[50];
    int id;
} Student;
void print_student(Student s) {
    printf("Name: %s, ID: %d\n", s.name, s.id);
}
int main() {
    Student s = {"Alice", 101};
    print_student(s);
    return 0;
}
// Expected Output: Name: Alice, ID: 101

```

7.2 #pragma Directive

Explanation:

The `#pragma` directive gives special instructions to the compiler. For example, `#pragma pack` controls how a structure's data is stored in memory, reducing extra space (padding) between items.

Example:

```

#include <stdio.h>
#pragma pack(push, 1)
struct Packed {
    int x;
    char c;
};
#pragma pack(pop)
int main() {
    struct Packed p = {1, 'A'};
    printf("Size: %zu\n", sizeof(p));
    return 0;
}
// Expected Output: Size: 5 (or smaller, depending on system)

```

7.3 Array of Structures

Explanation:

You can make an array where each element is a structure, like a list of students. This is useful for storing multiple records.

Example:

```

#include <stdio.h>
typedef struct {
    char name[50];
    int id;
} Student;
int main() {
    Student students[2] = {"Bob", 102}, {"Charlie", 103};
    for (int i = 0; i < 2; i++) {
        printf("Student %d: %s\n", students[i].id, students[i].name);
    }
    return 0;
}
// Expected Output:
// Student 102: Bob
// Student 103: Charlie

```

7.4 Problem Solving Using Structures and Functions

Explanation:

Structures and functions can work together to solve problems, like sorting a list of students by their IDs.

Example:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct {
    char name[50];
    int id;
} Student;
int compare_students(const void *a, const void *b) {
    return ((Student*)a)->id - ((Student*)b)->id;
}
int main() {
    Student students[3] = {"Charlie", 103}, {"Alice", 101}, {"Bob", 102};
    qsort(students, 3, sizeof(Student), compare_students);
    for (int i = 0; i < 3; i++) printf("%d: %s\n", students[i].id, students[i].name);
    return 0;
}
// Expected Output:
// 101: Alice
// 102: Bob
// 103: Charlie

```

7.5 Sorting and Searching: Array of Structures/Pointers to Structures

Explanation:

You can sort or search an array of structures or an array of pointers to structures. For example, you can search for a student by ID.

Example:

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char name[50];
    int id;
} Student;
int main() {
    Student s1 = {"Alice", 101}, s2 = {"Bob", 102};
    Student *ptrs[] = {&s1, &s2, NULL};
    for (int i = 0; ptrs[i]; i++) {
        if (ptrs[i]->id == 101) printf("Found: %s\n", ptrs[i]->name);
    }
    return 0;
}
// Expected Output: Found: Alice
```

7.6 Bit Fields

Explanation:

Bit fields let you store data using a specific number of bits in a structure. This saves memory when you only need a few bits for each item, like flags (on/off).

Example:

```
#include <stdio.h>
struct Flags {
    unsigned int on : 1; // Uses 1 bit
    unsigned int error : 1;
};
int main() {
    struct Flags f = {1, 0};
    printf("On: %d\n", f.on);
    return 0;
}
// Expected Output: On: 1
```

7.7 Unions

Explanation:

A union is like a structure, but all its items share the same memory space. Only one item can be used at a time, which saves memory when you need to store different types of data.

Example:

```
#include <stdio.h>
union Data {
    int i;
    float f;
};
int main() {
    union Data d;
    d.i = 65;
    printf("As char: %c\n", *(char*)&d.i);
    return 0;
}
// Expected Output: As char: A
```

7.8 Enumerations

Explanation:

An enumeration (`enum`) lets you create a list of named numbers. It makes your code easier to read by using names instead of numbers.

Example:

```
#include <stdio.h>
enum Day {MON, TUE, WED};
int main() {
    enum Day today = TUE;
    printf("Day: %d\n", today);
    return 0;
}
// Expected Output: Day: 1
```

Module 8: File Handling

8.1 File Handling Functions and Error Handling

Explanation:

File handling lets your program read from or write to files on your computer. You use a `FILE` pointer to work with files, and functions like `fopen` to open them, `fclose` to close them, and others to read or write data. Always check for errors (e.g., if the file doesn't exist).

Example:

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("data.txt", "w");
    if (!fp) {
        printf("Error opening file\n");
        return 1;
    }
    fprintf(fp, "Hello\n");
    fclose(fp);

    fp = fopen("data.txt", "r");
    char line[100];
    fgets(line, 100, fp);
    printf("Read: %s", line);
    fclose(fp);
    return 0;
}
// Expected Output: Read: Hello
```

Binary File:

```
#include <stdio.h>
typedef struct { int id; } Record;
int main() {
    Record r = {101};
    FILE *fp = fopen("data.bin", "wb");
    fwrite(&r, sizeof(Record), 1, fp);
    fclose(fp);

    Record r2;
    fp = fopen("data.bin", "rb");
    fread(&r2, sizeof(Record), 1, fp);
    printf("ID: %d\n", r2.id);
    fclose(fp);
    return 0;
}
// Expected Output: ID: 101
```

8.2 File I/O Using Redirection

Explanation:

Redirection lets you use files as input or output for your program without changing the code. For example, you can make `scanf` read from a file instead of the

keyboard.

Example:

Run: `./program < input.txt > output.txt`

- `input.txt` provides data for `scanf`.
- Output goes to `output.txt` instead of the screen.

Sample Program:

```
#include <stdio.h>

int main() {
    int num;
    scanf("%d", &num);
    printf("Number: %d\n", num);
    return 0;
}

// input.txt contains: 42
// Expected Output (in output.txt): Number: 42
```

8.3 Searching and Sorting in Files Using Array of Pointers

Explanation:

You can read data from a file into an array of pointers (e.g., an array of strings) and then search or sort it. This is useful for tasks like finding a word in a file.

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LINES 100
#define MAX_LEN 256
int main() {
    FILE *fp = fopen("data.txt", "w");
    fprintf(fp, "Hello World\nThis is a test\n");
    fclose(fp);

    fp = fopen("data.txt", "r");
    char *lines[MAX_LINES];
    int count = 0;
    char buffer[MAX_LEN];
    while (fgets(buffer, MAX_LEN, fp) && count < MAX_LINES) {
        buffer[strcspn(buffer, "\n")] = '\0';
        lines[count] = (char*)malloc(strlen(buffer) + 1);
        strcpy(lines[count], buffer);
        count++;
    }
    fclose(fp);
    for (int i = 0; i < count; i++) {
        if (strstr(lines[i], "Hello")) printf("Line %d: %s\n", i + 1, lines[i]);
        free(lines[i]);
    }
    return 0;
}

// Expected Output: Line 1: Hello World
```

Multiple Files: Copy between files.

```
#include <stdio.h>
int main() {
    FILE *in = fopen("in.txt", "w");
    fprintf(in, "Sample text\n");
    fclose(in);

    in = fopen("in.txt", "r");
    FILE *out = fopen("out.txt", "w");
    char buffer[256];
    while (fgets(buffer, 256, in)) {
        fputs(buffer, out);
    }
    fclose(in);
    fclose(out);

    out = fopen("out.txt", "r");
    fgets(buffer, 256, out);
    printf("Copied: %s", buffer);
    fclose(out);
    return 0;
}
// Expected Output: Copied: Sample text
```

Module 9: Build Process and Preprocessor

9.1 Header File Creation and Use

Explanation:

Header files (`.h`) let you share code between different files. They usually contain function declarations, and you write the actual code in a `.c` file. This keeps your program organized.

Example:

- `mymath.h`:

```
#ifndef MYMATH_H
#define MYMATH_H
int add(int a, int b);
#endif
```

- `mymath.c`:

```
#include "mymath.h"
int add(int a, int b) { return a + b; }
```

- `main.c`:

```
#include <stdio.h>
#include "mymath.h"
int main() {
    printf("Sum: %d\n", add(2, 3));
    return 0;
}
// Expected Output: Sum: 5
```

Compile: `gcc main.c mymath.c -o program .`

9.2 Environment Variables

Explanation:

Environment variables are settings on your computer, like your username. You can read them in C using `getenv` to make your program adapt to the system.

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    char *user = getenv("USER");
    if (!user) user = getenv("USERNAME");
    printf("Hello, %s!\n", user ? user : "Unknown");
    return 0;
}
// Expected Output: Hello, (your username or "Unknown")!
```

9.3 Preprocessor Directives and Conditional Compilation

Explanation:

The preprocessor runs before your code is compiled and can change it. Directives start with `#` :

- `#include` : Adds a file (like `stdio.h`).
- `#define` : Creates a constant or macro.
- `#ifdef` / `#endif` : Includes code only if a condition is true (e.g., for debugging).
This helps make your program work on different systems (portability).

Examples:

- **Debugging:**

```
#include <stdio.h>
#define DEBUG
int main() {
    #ifdef DEBUG
        printf("Debug mode\n");
    #endif
    return 0;
}
// Expected Output: Debug mode
```

- **Portability:**

```
#include <stdio.h>
void clear() {
    #ifdef _WIN32
        printf("Windows clear\n");
    #else
        printf("Unix clear\n");
    #endif
}
int main() {
    clear();
    return 0;
}
// Expected Output: (Depends on system; e.g., "Unix clear" on Linux)
```

Conclusion

This guide has explained each C programming topic with simple explanations, examples, and expected outputs. To get better, try writing your own programs, like a calculator or a to-do list. Practice is the key to mastering C!